PATRICIA MBOMA KASHWEKA

2021527433

READING ASSIGNMENT: INTRO INTO WEB PROGRAMMING

REFLECTION QUESTIONS

1. From the evolution of web technologies, which innovation do you think has had the greatest impact on modern web development? Why?

   The most relevant Lehman's Law nowadays is the Law of Continuing Change, which states that software must be constantly adjusted or else it becomes progressively less valuable. Nowadays, technology rapidly evolves—operating systems, hardware, user needs and security requirements evolve on a periodic basis. Unless software is periodically updated, it can become obsolete or insecure. For instance, mobile apps have to be constantly updated so that they become compatible with the newer operating system versions and user sentiments. Otherwise, they lose users or become security vulnerabilities. This action demonstrates that software needs to be viewed as a dynamic product that needs to evolve with its environment. It also encourages good designing and coding so that updates and merges are easier. Software makers and companies with knowledge of this law plan ahead for long-term maintenance. Software adaptation is no longer an option it's required for survival, competitiveness, and customer satisfaction—during an era of constant digital evolution. The law is reality in today's software development and the fact that constant investment is needed decades after the rollout.

2. Explain the importance of HTTPS in securing web communication. Provide an example of a website that implements HTTPS effectively.

   Maintenance costs more than development since it entails occasional updates, bug fixing, security patches, and adaptation to evolving requirements over time. Maintenance, as noted by the MadDevs article, accounts for 60–90% of the software system's total cost of ownership. Maintenance has concealed costs such as dealing with legacy code, poor documentation, and technical debt management. As opposed to development, a one-off activity with a definitive objective and deadline, maintenance is ongoing and unpredictable. In maintaining a legacy system, for instance, programmers might have to devote more time to studying legacy code or debugging issues that make whole systems crash. Patches also must be more thoroughly tested, particularly if users rely on the software to carry out vital tasks. If not undertaken carefully, a minor modification can break existing functionality. In addition to this, changing technologies and users' expectations require features to be updated continuously, further contributing to maintenance expenses. All these aspects complicate software maintenance and make it a time-taking process despite being undervalued in contrast to the original development process.

3. Describe the Client-Server Model in your own words. How does it work in a real-world scenario like social media or online banking

   When a company ceases to support software, it exposes itself to security threats, performance degradation, and loss of customer trust. Obsolete software can be incompatible with new

systems and may not suit user requirements. Eventually, it results in user discontent, lower productivity, and financial losses. One of the most tried and tested real-world instances is the 2017 WannaCry ransomware attack on computers running on outdated versions of Windows that did not have essential security patches. Most companies and hospitals were severely affected since they had not upgraded their systems. This resulted in massive data loss, downtime, and millions of dollars in losses. Beyond security, outdated software also equates to missed opportunities—companies can't respond to user input, improve features, or adapt to industry innovations. The older software gets, the more difficult it is to integrate with new systems, and increasingly expensive to find developers to build for it. The longer it takes before a system is maintained, the harder and more costly it will be to get it up to par. Ultimately, not keeping up with it will turn software into a liability, rather than an asset.

4. What are some common HTTP status codes, and how do they affect website performance and troubleshooting

Git provides a number of improvements over older version control systems such as CVS or SVN. Perhaps its greatest strength is the fact that it is distributed. Every developer receives a complete copy of the repository and can thus work independently and offline, synching later. This makes it both more flexible and less dependent upon a central server. Git also manages branching and merging more effectively than older systems. Developers are allowed to make light feature or bug fix branches, test them separately, and merge them into the main codebase without interfering with other individuals' work. It allows parallel development and collaboration, a requirement in today's software teams. It is also extremely fast and efficient with large codebases. It maintains data integrity through its utilization of cryptographic hashes. Two tools, GitHub and GitLab, seamlessly integrate with Git to provide easy collaboration, issue management, and automated workflows (CI/CD). The wide use of Git and its strong community support also ensure there is always support, tutorials, and integrations available at hand. In conclusion, Git's flexibility, performance, and ecosystem make it the tool of choice for modern software development in open-source and enterprise environments alike.